

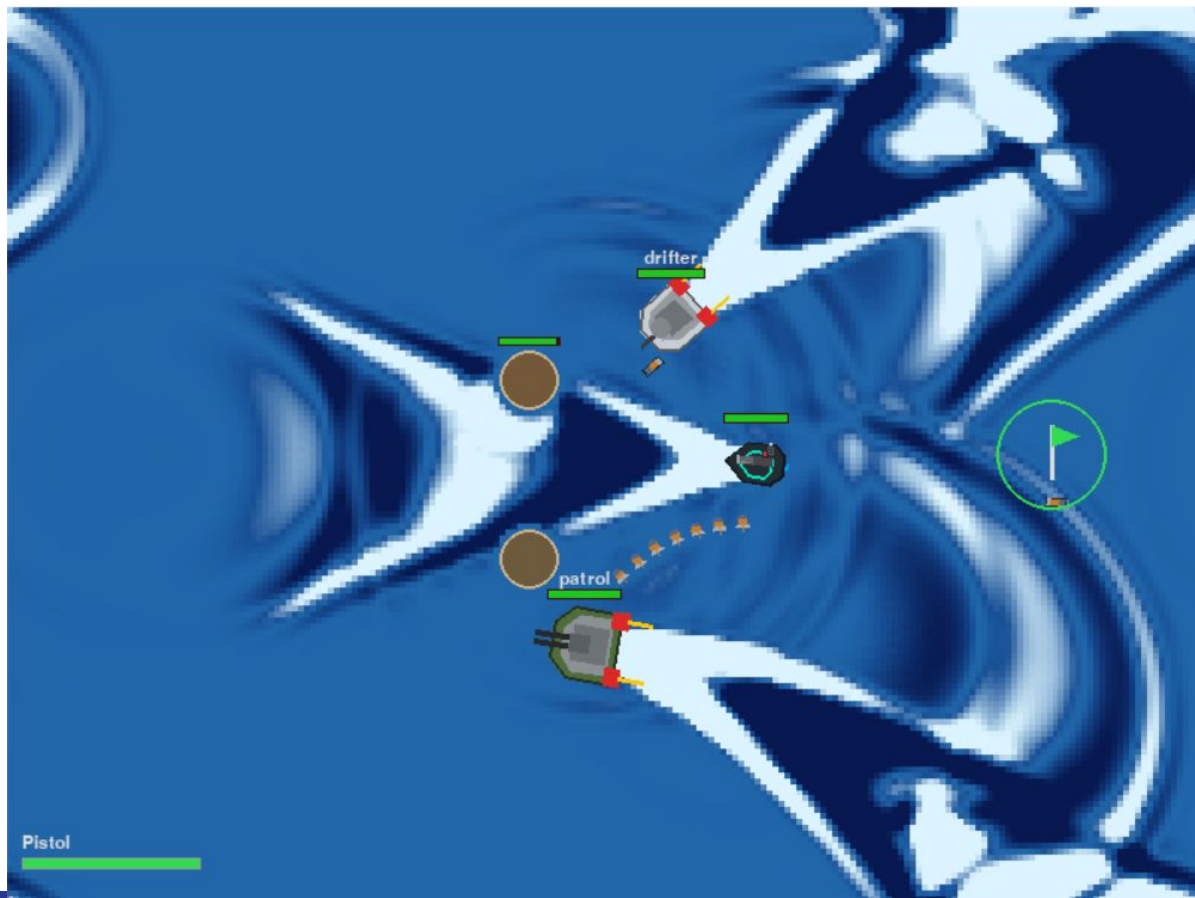
Boat Simulator

By: Dean Francis E. Gomez, Nicholas Gunawan, Zhixian Lu

Intro

Goal:

Create a fun game that integrates a fluid simulation and basic elements of a 2D top down shooter game



Demo
Levels
1-3

BOAT SIMULATOR

WASD/arrows to move • Hold LMB to fire

PLAY

WEAPONS

QUIT

Level 4

SELECT LEVEL

ESC / BACK to return to main menu

Open Water

The Gauntlet

Crossfire

The Maze

LOCKED

BACK

Level 5

SELECT LEVEL

ESC / BACK to return to main menu

Open Water

The Gauntlet

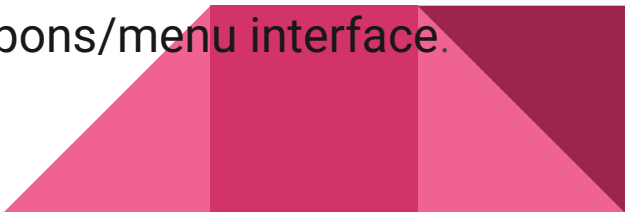
Crossfire

The Maze

No Return

BACK

Features

- Player controls with keyboard for movement and mouse for aiming and shooting
 - Simple visual and physical fluid effects, including boundaries and obstacles
 - 5 weapon types, with firing mechanics, cooldown handling, and a weapons_menu for selection
 - 6 enemies with different HP and weapons, with basic player tracking, and collision responses.
 - 5 levels, level selection, and win/lose conditions
 - Modular entity classes (player, enemies, bullets, items) and a central game loop managing updates and rendering
 - Main menu, in-game HUD, pause/options, and a weapons/menu interface.
- 

Structure of Code

```
main.py (entry point / flow control)
├── menu.py
│   └── water.py
├── level_select.py
│   └── water.py
├── levels.py
│   ├── water.py
│   └── weapon.py
├── game.py (main loop / physics / collision)
│   ├── water.py
│   ├── entity.py
│   ├── weapon.py
│   ├── enemy.py
│   │   ├── entity.py
│   │   ├── weapon.py
│   │   └── water.py
│   ├── bullet.py
│   └── player_visual.py
├── enemy.py (imported for functions)
│   ├── entity.py
│   ├── weapon.py
│   └── water.py
└── weapons_menu.py
    ├── water.py
    └── weapon.py
```

Level Select/Design

Buttons

```
class Button:
    def __init__(self, x: int, y: int, w: int, h: int,
                 text: str, unlocked: bool = True):
        self.rect = pygame.Rect(x, y, w, h)
        self.text = text
        self.unlocked = unlocked

    def draw(self, surface: pygame.Surface, font: pygame.font.Font,
            mx: int, my: int):
        hov = self.unlocked and self.rect.collidepoint(mx, my)
        color = DARK_GRAY if not self.unlocked else (BLUE_HOV if hov else BLUE)
        pygame.draw.rect(surface, color, self.rect, border_radius=8)
        pygame.draw.rect(surface, BLACK, self.rect, 2, border_radius=8)
        label = self.text if self.unlocked else "LOCKED"
        txt = font.render(label, True, WHITE)
        surface.blit(txt, (self.rect.centerx - txt.get_width() // 2,
                          self.rect.centery - txt.get_height() // 2))

    def is_clicked(self, event: pygame.event.Event) -> bool:
        return (self.unlocked
                and event.type == pygame.MOUSEBUTTONDOWN
                and event.button == 1
                and self.rect.collidepoint(event.pos))
```

Level Select Screen

```
class LevelSelect:
    """Level-selection screen. Call run() → returns 0-based level index or -1."""

    def __init__(self, level_names: list,
                  levels_unlocked: int = 1,
                  width: int = WIDTH, height: int = HEIGHT):
        if not pygame.get_init():
            pygame.init()
        self.screen = pygame.display.get_surface() or pygame.display.set_mode((width, height))
        self.clock = pygame.time.Clock()
        self._font = pygame.font.SysFont(None, 36)
        self._title_font = pygame.font.SysFont(None, 60)
        self._width = width
        self._height = height

        positions = list(self._layout(level_names, width, height))
        self._buttons = [
            Button(x, y, 200, 70, name, i < levels_unlocked)
            for i, (name, x, y) in enumerate(positions)
        ]
        self._back = Button(30, height - 70, 120, 48, "BACK", unlocked=True)
```

Event Loop/Reusability

```
def run(self) -> int:
    """Event loop. Returns 0-based level index, or -1 on back/quit."""
    while True:
        mx, my = pygame.mouse.get_pos()

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                return -1
            if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
                return -1
            if self._back.is_clicked(event):
                return -1
            for i, btn in enumerate(self._buttons):
                if btn.is_clicked(event):
                    return i

        self.screen.fill(BG)

        title = self._title_font.render("SELECT LEVEL", True, (200, 230, 255))
        self.screen.blit(title, (self._width // 2 - title.get_width() // 2, 120))

        sub = self._font.render(
            "ESC / BACK to return to main menu", True, (100, 140, 200))
        self.screen.blit(sub, (self._width // 2 - sub.get_width() // 2, 168))

        for btn in self._buttons:
            btn.draw(self.screen, self._font, mx, my)

        self._back.draw(self.screen, self._font, mx, my)

        pygame.display.flip()
        self.clock.tick(FPS)
```

Levels 1-3 Simple

```
from dataclasses import dataclass
from water import GRID_W, GRID_H
from weapon import Pistol, MachineGun, Sniper, Shotgun, Bazooka

@dataclass
class LevelDef:
    name: str
    player_start: tuple
    flag_pos: object # (x, y) or None for survival levels
    obstacles: list # dicts: {"x", "y", "radius", "color"(opt)}
    enemies: list # dicts: {"type", "x", "y", "cx"(opt), "cy"(opt)}
    splashes: list # (gx, gy, amp, radius)
    survival: bool = False
    survival_secs: float = 120.0
    current_force: float = 80.0 # px/s2 leftward (applied to player only)
    scroll_speed: float = 4.0 # grid cells/s scrolled left visually
    player_weapon: object = None # Weapon instance given to the player
    enemy_budget: int = 0
    spawn_pool: list = None
    topology: str = "torus" # "torus" = wrap edges; "bounded" = wall bounce
```


Level 4 - The Maze

```
# Maze layout (1 = wall, 0 = path)
maze = [
    [1,1,1,1,1,1,1,1,1,1,1,1,1,1,1],
    [1,0,0,0,1,0,0,0,0,0,1,0,0,0,1],
    [1,0,1,0,1,0,1,1,1,0,1,0,1,1,0,1],
    [1,0,1,0,0,0,1,0,0,0,1,0,0,1,0,1],
    [1,0,1,1,1,0,1,0,1,1,1,1,0,1,0,1],
    [1,0,0,0,1,0,0,0,0,0,0,1,0,1,0,1],
    [1,1,1,0,1,1,1,1,1,1,0,1,0,1,0,1],
    [1,0,0,0,0,0,0,0,0,1,0,0,0,1,0,1],
    [1,0,1,1,1,1,1,1,0,1,1,1,0,1,0,1],
    [1,0,0,0,0,0,0,0,1,0,0,0,1,0,0,1],
    [1,1,1,1,1,1,0,1,1,1,0,1,1,1,0,1],
    [1,0,0,0,0,1,0,0,0,1,0,0,0,1,0,1],
    [1,0,1,1,0,1,1,1,0,1,1,1,0,1,0,1],
    [1,0,0,1,0,0,0,1,0,0,0,1,0,0,0,1],
    [1,0,0,0,0,1,0,0,0,1,0,0,0,1,0,1],
    [1,1,1,1,1,1,1,1,1,1,1,1,1,1,1],
]
```

```
# — Level 4: The Maze (procedural) —
import random

def _generate_maze(width, height):
    # Ensure odd dimensions
    if width % 2 == 0:
        width += 1
    if height % 2 == 0:
        height += 1

    maze = [[1 for _ in range(width)] for _ in range(height)]

    def carve(x, y):
        directions = [(2,0), (-2,0), (0,2), (0,-2)]
        random.shuffle(directions)

        for dx, dy in directions:
            nx, ny = x + dx, y + dy

            if 1 <= nx < width - 1 and 1 <= ny < height - 1:
                if maze[ny][nx] == 1:
                    maze[ny][nx] = 0
                    maze[y + dy//2][x + dx//2] = 0
                    carve(nx, ny)

    maze[1][1] = 0
    carve(1, 1)

    # Ensure start and end are open
    maze[1][1] = 0
    maze[-2][-2] = 0

    return maze
```

Level 4 - The Maze

```
def _maze_to_obstacles(maze, cell_size=50):
    obstacles = []
    for r in range(len(maze)):
        for c in range(len(maze[0])):
            if maze[r][c] == 1:
                x = c * cell_size + cell_size // 2
                y = r * cell_size + cell_size // 2

                obstacles.append({
                    "x": x,
                    "y": y,
                    "radius": int(cell_size * 0.45)
                })
    return obstacles

def _cell_to_world(r, c, cell_size=50):
    return (
        c * cell_size + cell_size // 2,
        r * cell_size + cell_size // 2
    )

def _get_open_cells(maze):
    cells = []
    for r in range(len(maze)):
        for c in range(len(maze[0])):
            if maze[r][c] == 0:
                cells.append((r, c))
    return cells
```

```
cell_size = 50
cols = 15 # must be odd
rows = 11 # must be odd

maze = _generate_maze(cols, rows)
maze_obstacles = _maze_to_obstacles(maze, cell_size)

# Place enemies only in valid path cells
open_cells = _get_open_cells(maze)
enemies = []

for r, c in random.sample(open_cells, min(6, len(open_cells))):
    x, y = _cell_to_world(r, c, cell_size)
    enemies.append({
        "type": random.choice(["chase", "patrol", "drift"]),
        "x": x,
        "y": y
    })

level4 = LevelDef(
    name = "The Maze",
    player_start = (cell_size + 40, cell_size + 40), # near (1,1)
    flag_pos = (
        (cols - 2) * cell_size,
        (rows - 2) * cell_size
    ),
    obstacles = maze_obstacles,
    enemies = enemies,
    splashes = [],
    player_weapon = Shotgun(),
)
```

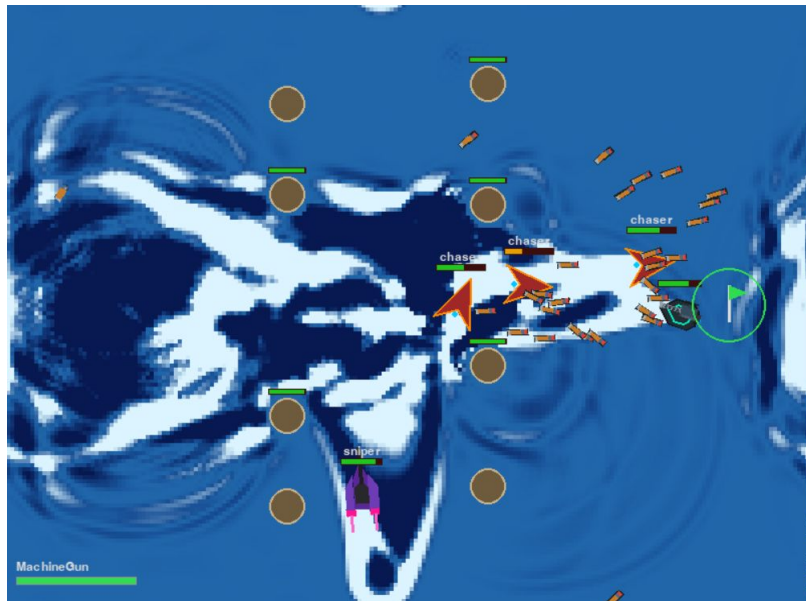
Level 5 - Time

```
# — Level 5: River of No Return (survival) —
level5 = LevelDef(
    name          = "No Return",
    player_start  = (650, 300),
    flag_pos      = None,          # win by surviving
    obstacles     = [],
    enemies       = [
        {"type": "chase", "x": 400, "y": 300},
        {"type": "chase", "x": 400, "y": 300},
        {"type": "chase", "x": 400, "y": 300},
        {"type": "chase", "x": 400, "y": 300},
        {"type": "chase", "x": 400, "y": 300},
        {"type": "chase", "x": 400, "y": 300},
        {"type": "chase", "x": 400, "y": 300},
        {"type": "chase", "x": 400, "y": 300},
        {"type": "patrol", "x": 300, "y": 200, "cx": 300, "cy": 200},
        {"type": "snipe", "x": 600, "y": 100},
    ],
    splashes      = [
        (GRID_W * 3 // 4, GRID_H // 2, 0.02, 4),
    ],
    survival      = True,
    survival_secs = 120.0,
    current_force = 90.0,
    scroll_speed  = 3.0,
    player_weapon = Bazooka(),
)
```



Fluid Simulation and Entity Interactions

Fluid Simulation and Visuals



```
# --- window & grid ---
WIDTH, HEIGHT = 800, 600           # screen / viewport size
WORLD_W, WORLD_H = WIDTH, HEIGHT   # world matches screen (no scrolling)
GRID_W, GRID_H = 100*2, 75*2
CELL_W = WORLD_W // GRID_W         # 8 px per grid cell
CELL_H = WORLD_H // GRID_H         # 8 px per grid cell
FPS = 60

# --- Wave physics ---
WAVE_SPEED = 0.4 # CFL: must stay below 1/sqrt(2) ≈ 0.707
DAMPING = 0.99 # faster energy dissipation
SPLASH_AMP = 0.005
SPLASH_R = 10

# Numerical stabilization
VISCOSITY = 0.1 # Laplacian-based diffusion to damp short wavelengths
# Hard clamp - must stay above any single-frame injection to avoid h vs h_prev divergence
MAX_H = 0.05

# --- Entity-water coupling ---
WAVE_GRAD_K = 7000.0 # wave gradient force on entities (was 0.01)
DRAG_K = 0.02 # 0.02
DISPLACE_AMP = 0.0001 # was 0.01 - large values caused constant saturation
RIPPLE_AMP = 0.0005 # was 0.012 - scaled down to stay well below MAX_H

# --- Rendering palette ---
COLOR_DEEP = np.array([8, 24, 80], dtype=np.float32)
COLOR_MID = np.array([24, 80, 150], dtype=np.float32)
COLOR_SHALLOW = np.array([60, 170, 230], dtype=np.float32)
COLOR_FOAM = np.array([220, 245, 255], dtype=np.float32)
COLOR_CRESCENT = np.array([200, 230, 255], dtype=np.float32)
```

WaterGrid Class

(stores and simulates fluid)

```
def __init__(self, grid_h: int = GRID_H, grid_w: int = GRID_W):  
    self.grid_h = grid_h  
    self.grid_w = grid_w  
    self.c2 = WAVE_SPEED ** 2  
    self.h = np.zeros((grid_h, grid_w), dtype=np.float32)  
    self.h_prev = np.zeros((grid_h, grid_w), dtype=np.float32)  
    self.fixed = np.zeros((grid_h, grid_w), dtype=bool)  
    # Precompute c^2 for wave equation  
    # Current height  
    # Previous height  
    # Obstacle mask
```



Fluid Simulation and Visuals

Laplacian (measures how curved a spot is compared to neighbors)

$$\nabla^2 h_{i,j} \approx h_{i+1,j} + h_{i-1,j} + h_{i,j+1} + h_{i,j-1} - 4h_{i,j}$$

Wave equation (discretized)

$$\frac{\partial^2 h}{\partial t^2} = c^2 \nabla^2 h$$

$$h^{t+1} = 2h^t - h^{t-1} + c^2 \nabla^2 h^t$$

```
@staticmethod
def _laplacian(h: np.ndarray) -> np.ndarray:
    return (
        np.roll(h, -1, axis=0) + np.roll(h, 1, axis=0) +
        np.roll(h, -1, axis=1) + np.roll(h, 1, axis=1) -
        4.0 * h
    )

"""Advance the wave simulation by one time step"""
def step(self):
    lap = self._laplacian(self.h)
    h_new = 2.0 * self.h - self.h_prev + self.c2 * lap

    # apply small laplacian-based viscosity to damp nyquist/short waves
    if VISCOSITY > 0.0:
        h_new -= VISCOSITY * lap

    # global damping
    h_new *= DAMPING

    # clamp extreme values to prevent numerical blowup
    if MAX_H is not None:
        np.clip(h_new, -MAX_H, MAX_H, out=h_new)

    h_new[self.fixed] = 0.0
    self.h_prev = self.h
    self.h = h_new
```

Entity-Water Coupling ()

```
class Entity:
    """Anything that lives in the water: player, enemy boat, or static obstacle."""
    x: float
    y: float
    vx: float = 0.0
    vy: float = 0.0
    mass: float = 1.0
    radius: float = 12.0
    color: tuple = (0, 200, 80)
    static: bool = False
    controllable: bool = False
    name: str = ""
    max_hp: float = 100.0
    hp: float = field(default=0.0)
    wave_grad_k: float = 1.0 # how much affected by water waves (1.0 = base)
    ax: float = field(default=0.0, repr=False)
    ay: float = field(default=0.0, repr=False)

    def __post_init__(self):
        if self.hp == 0.0:
            self.hp = self.max_hp

    @property
    def alive(self) -> bool:
        return self.hp > 0.0
```

```
def _couple_entities(self):
    dt = 1.0 / self.fps
    for e in self.entities:
        if e.static or not e.alive:
            continue
        gx = e.x / CELL_W; gy = e.y / CELL_H
        gr = e.radius / max(CELL_W, CELL_H)
        self.water.splash(gx, gy, -DISPLACE_AMP * e.mass, gr)
        spd = math.hypot(e.vx, e.vy)
        # Enemies create much weaker ripples to avoid saturating the grid
        ripple = RIPPLE_AMP * (1.0 if e.controllable else 0.12)
        if spd > 5.0:
            self.water.splash(gx, gy, ripple * spd * dt, max(gr * 0.6, 0.5))
        dx_w, dy_w = self.water.gradient_at(gx, gy)
        # Use per-entity wave coupling strength
        grad_k = WAVE_GRAD_K * e.wave_grad_k
        e.ax += -grad_k * dx_w / e.mass
        e.ay += -grad_k * dy_w / e.mass
        e.vx *= 1.0 - DRAG_K
        e.vy *= 1.0 - DRAG_K
```

Integration with Game Physics

```
def _integrate_entities(self):
    dt = 1.0 / self.fps
    for e in self.entities:
        if e.static or not e.alive:
            e.ax = e.ay = 0.0; continue
        e.vx += e.ax * dt; e.vy += e.ay * dt
        if self.survival and self._current_force > 0:
            # u
            e.x = (e.x + e.vx * dt) % self.world_w
            e.y = (e.y + e.vy * dt) % self.world_h
            if e.x < 0:
                e.x = float(self.world_w)
            elif e.x > self.world_w:
                e.x = float(self.world_w)
        elif self.topology == "bounded" or self.entity_barriers:
            e.x += e.vx * dt; e.y += e.vy * dt
            if e.x < e.radius:
                e.x = e.radius
                e.vx = abs(e.vx)
            elif e.x > self.world_w - e.radius:
                e.x = self.world_w - e.radius
                e.vx = -abs(e.vx)
            if e.y < e.radius:
                e.y = e.radius
                e.vy = abs(e.vy)
            elif e.y > self.world_h - e.radius:
                e.y = self.world_h - e.radius; e.vy = -abs(e.vy)
        else: # wrap both axes
            e.x = (e.x + e.vx * dt) % self.world_w
            e.y = (e.y + e.vy * dt) % self.world_h
        e.ax = 0.0; e.ay = 0.0
```



Weapons, Weapons
Menu, Player Visual,
and Enemies

Weapons Designs - Pistol/Machine Gun

```
# — Pistol —

class Pistol(Weapon):
    """Single accurate shot."""
    fire_rate = 1.0

    def _make_projectiles(self, x, y, angle):
        #perfectly aligns the velocity with where the player is aiming
        vx, vy = self._vel(1000.0, angle)
        return [Projectile(x=x, y=y, vx=vx, vy=vy,
                           damage=20.0, lifetime=2.5,
                           weapon_type="pistol",
                           color=(255, 255, 100))] #Sets 20 damage and a 2.5-second lifetime

# — Machine Gun —

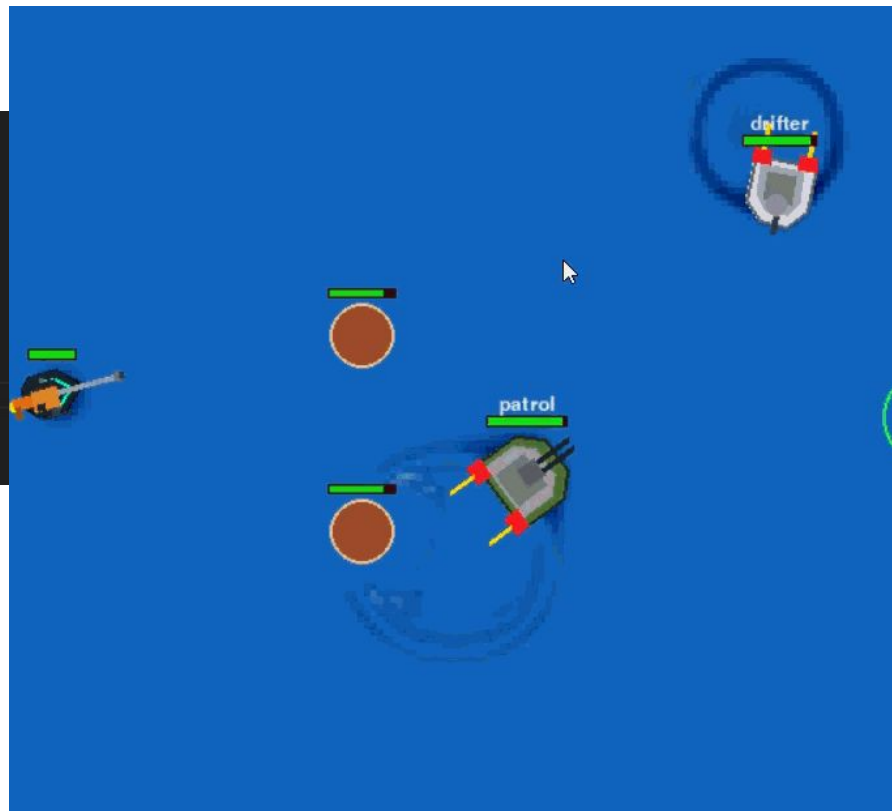
class MachineGun(Weapon):
    """Fast fire rate, inaccurate n less damage."""
    fire_rate = 8.0

    def _make_projectiles(self, x, y, angle):
        #18-degree spread into the velocity function,
        # uses the random.uniform function to crete spray effect
        vx, vy = self._vel(1120.0, angle, spread=math.radians(18))
        return [Projectile(x=x, y=y, vx=vx, vy=vy,
                           damage=8.0, lifetime=1.8,
                           weapon_type="machine_gun",
                           color=(255, 180, 50))]
```



Weapons Designs - Sniper

```
# — Sniper —  
  
class Sniper(Weapon):  
    """Slow rate, very fast bullet, high damage, pierces enemies."""  
    fire_rate = 0.4  
  
    def _make_projectiles(self, x, y, angle):  
        vx, vy = self._vel(2800.0, angle)  
        return [Projectile(x=x, y=y, vx=vx, vy=vy,  
                           damage=55.0, lifetime=3.0,  
                           pierce=True, #gives piercing effect  
                           weapon_type="sniper",  
                           color=(180, 80, 255))]  
  
#sniper skips this  
if not proj.pierce:  
    killed = True  
    if proj.explodes:  
        self._do_explosion(proj)  
    break
```



Weapons Designs - Bazooka

```
# — Bazooka —

class Bazooka(Weapon):
    """Slow projectile that explodes on impact or after a fuse timer."""
    fire_rate = 0.3

    def _make_projectiles(self, x, y, angle):
        vx, vy = self._vel(640.0, angle)
        return [Projectile(x=x, y=y, vx=vx, vy=vy,
                           damage=0.0, lifetime=5.0, #explosion oes damage
                           explodes=True,
                           explosion_radius=320.0,
                           explosion_damage=600.0,
                           fuse=3.0,
                           weapon_type="bazooka",
                           color=(255, 100, 30))]

    def _do_explosion(self, proj: Projectile):
        """Apply AoE damage, water splash, and schedule a visual flash."""
        #loops through every entity in the game
        for e in self.entities:
            if not e.alive or id(e) in proj.hits:
                continue
            #calculates the distance between the explosion's center and the entity
            d = self._torus_dist(proj.x, proj.y, e.x, e.y)
            if d < proj.explosion_radius: #If the entity is within blast radius, calculates the damage
                #closer to the center, more damage
                dmg = proj.explosion_damage * max(0.0, 1.0 - d / proj.explosion_radius)
```



Weapons Designs - Shotgun

```
# Shotgun
> shot Aa ab .* 9 of 14 ↑ ↓ ☰

class Shotgun(Weapon):
    """7 pellets in a cone; damage falls off with distance."""
    fire_rate = 0.8
    _pellets = 7 #shoots 7 individual bullets
    _cone = math.radians(22) #shot evenly across 22 degree cone
    _base_dmg = 30.0
    _range = 1120.0

    def _make_projectiles(self, x, y, angle):
        projs = []
        for i in range(self._pellets): #loop 7 times and creates 7 bullets
            #Creates value between 0.0 - 1.0 base on index: 0, 0.16, 0.33, 0.5, 0.66, 0.83, 1.0
            t = i / (self._pellets - 1) if self._pellets > 1 else 0.5
            #center angle + cone angle
            a = angle + self._cone * (t * 2 - 1) #creates -1.0 to 1.0 range create spread effect
            vx, vy = self._vel(960.0, a)
            projs.append(Projectile(
                x=x, y=y, vx=vx, vy=vy,
                damage=self._base_dmg, lifetime=1.2,
                shotgun=True, shotgun_range=self._range,
                origin_x=x, origin_y=y,
                weapon_type="shotgun",
                color=(255, 140, 60),
            ))
        return projs
```



```
if self._torus_dist(proj.x, proj.y, e.x, e.y) <= e.radius:
    # calculate how much shotgun damage does base on distance travelled
    if proj.shotgun:
        traveled = self._torus_dist(proj.origin_x, proj.origin_y,
                                     proj.x, proj.y) #calculate distance between starting point to current point
        #traveled / proj.shotgun_range: calculates the % of the max range the bullet traveled
        #1.0 - ...: 100% - (% bullet traveled)
        #max(0.1,...: ensure damage never drops 10% of base damage
        dmg = proj.damage * max(0.1, 1.0 - traveled / proj.shotgun_range)
```

Weapons Menu - Carousel Mechanism

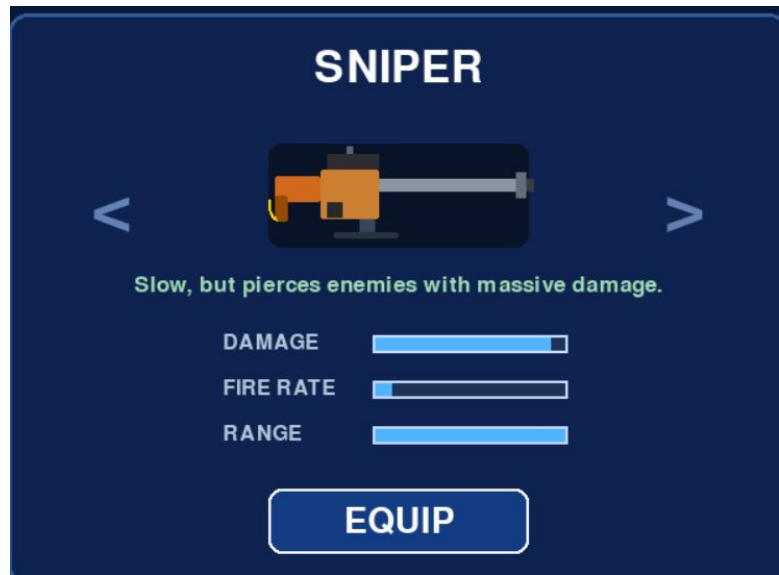
```
#tracks the currently displayed item
self.view_index = 0
for i, w in enumerate(self.weapon_data):
    if w["id"] == self.current_weapon:
        self.view_index = i
        break

# Modulo handles negative numbers by wrapping them around from the end of the list
if event.key == pygame.K_LEFT:
    self.view_index = (self.view_index - 1) % len(self.weapon_data)
# When the index reaches the length of list, index becomes 0, going back to
if event.key == pygame.K_RIGHT:
    self.view_index = (self.view_index + 1) % len(self.weapon_data)
while True:
    mx, my = pygame.mouse.get_pos()
    # self.view_index to fetch only the active weapon
    # weapon object renders weapon name, design, stat bars, and desc
    weapon = self.weapon_data[self.view_index]
    is_equipped = (weapon["id"] == self.current_weapon)
```



Weapons Menu - Weapon Design and Stat Bar

```
elif w_id == "sniper":  
    # self.screen : the Surface object where rectangle will render  
    # (210, 105, 30) : gives brown-orange color  
    # cx - 95, cy - 95, 35, 20 : positioning from center horizontally and vertically, width, height  
    # border_radius=2 : give rounded corners to rectangle  
    pygame.draw.rect(self.screen, (210, 105, 30), (cx - 95, cy - 95, 35, 20), border_radius=2)  
    #draw stat bar according to weapon stats  
    def _draw_stat_bar(self, x, y, label, bar_value, max_bar=10):  
        txt = self._stat_font.render(label, True, (180, 200, 220))  
        self.screen.blit(txt, (x, y))  
        bar_x, bar_w, bar_h = x + 115, 150, 14  
        #fix rectangle acts as empty "well" for stat bar (parameter) y: Any  
        pygame.draw.rect(self.screen, (bar_x, y + 2, bar_w, bar_h))  
        # fills the fix rectangle. multiplying the maximum bar width by the ratio of the current value to the maximum value  
        pygame.draw.rect(self.screen, (80, 180, 255), (bar_x, y + 2, int(bar_w * (bar_value / max_bar)), bar_h))  
        #gives white outer lining  
        pygame.draw.rect(self.screen, (200, 220, 255), (bar_x, y + 2, bar_w, bar_h), 2)
```



Weapons Menu

SELECT LEVEL

ESC / BACK to return to main menu

Open Water

The Gauntlet

Crossfire

The Maze

No Return

BACK



Enemies

```
# Import weapon classes from weapon file to enemy file  
from weapon import Weapon, Projectile, Pistol, MachineGun, Sniper, Bazooka, Shotgun
```

```
# Creating enemy with pistol weapon  
def make_drift(x: float, y: float) -> Enemy:  
    e = Entity(x=x, y=y, mass=1.5, radius=22,  
              color=(220, 60, 60), max_hp=2400, name="drifter")  
    # The enemy is created and explicitly handed a Pistol() from weapon.py  
    return Enemy(entity=e, weapon=Pistol(), ai_type="drift")
```

```
# Firing logic  
if self.weapon.ready: # Checks if the weapon is off cooldown  
    return self.weapon.fire(e.x, e.y, angle_to) # Pulls the trigger
```



Player Visual - Base and Weapon Design

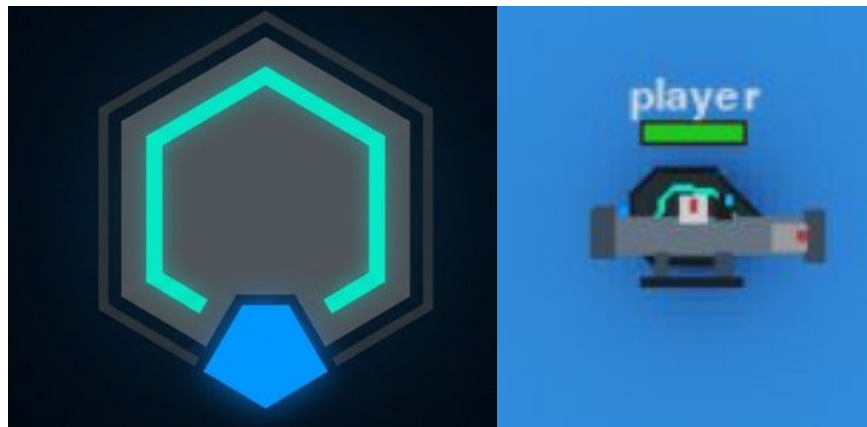
```
wt = (weapon_type or "pistol").lower()

if wt == "pistol":
    # Draws a small, simple set of gray rectangles
    draw_rotated_rect((70, 70, 75), -20, -2.5, 25, 12.5)
    ...
elif wt == "sniper":
    # Draws a long barrel and a brown stock
    draw_rotated_rect((210, 105, 30), -47.5, 1.25, 17.5, 10)
    ...
elif wt == "bazooka":
    # Draws a massive barrel and circular scope
    draw_rotated_rect((100, 115, 130), -42.5, 1.25, 85, 15)
    ...

#To be called for specific weapon and glue rectangle
def draw_rotated_rect(color, lx, ly, w, h, width=0):
    """Draw a rotated rectangle as a polygon."""
    corners = [
        (lx, ly),
        (lx + w, ly),
        (lx + w, ly + h),
        (lx, ly + h)
    ]
    rotated = [weapon_rot_p(px, py) for px, py in corners]
    pygame.draw.polygon(screen, color, rotated, width)
```

```
# define shape using 2D coordinates (x,y)
hull_points = [(20, 0), (-15, 12), (-10, 0), (-15, -12)]
rotated_hull = [rot_p(px, py) for px, py in hull_points]
# display surface where shape will appear; RGB color;
# border perfectly matches the shape drawn; width argument
pygame.draw.polygon(screen, (50, 150, 255), rotated_hull)
pygame.draw.polygon(screen, (20, 100, 200), rotated_hull, 2)

cabin_points = [(5, -6), (5, 6), (-8, 6), (-8, -6)]
rotated_cabin = [rot_p(px, py) for px, py in cabin_points]
pygame.draw.polygon(screen, (100, 110, 120), rotated_cabin)
pygame.draw.polygon(screen, (60, 70, 80), rotated_cabin, 2)
```



How AI helped and where did it fail?

- A large part of the code was AI generated
- A lot of the integration was done by AI
- AI helped with syntax and logic
-
- There were many times the fluid simulation failed and caused numerical instability
- Many times the water gradients sped up the enemies indefinitely
- (fixed by added viscosity and clamping of water height)
- Consistency issues, and difficulty integrating code together
- Settings and balancing the game

Questions?